Լաբորատոր աշխատանք 11

Դասեր C++ լեզվում · «Սրճարանի հաշիվներ» · N14

Կատարող՝ Hakob Avagyan · 2026

Աշխատանքի նպատակ

Ամրապնդել դասի հետ աշխատանքը՝ կոնստրուկտորում տվյալների ստուգում, օբյեկտի փոխանցում հղումով և արժեքով, օբյեկտի վերադարձ ֆունկցիայից և բնօրինակի անփոփոխության ստուգում։

Խնդրի պահանջը

Ստեղծել `CafeBills` դաս՝ սեղանների պատվերների գումարների զանգվածով և `p` սպասարկման տոկոսով։ Վերջնական հաշիվը՝ `amount · (1 + p/100)`։ Թույլատրելի արժեքներ՝ գումարները > 0, 0 ≤ p ≤ 100։ Հաշվել՝

- վերջնական հաշիվների գումարը,
- ամենափոքր վերջնական հաշվով սեղանի ինդեքսը (0-ից),
- 20000 դրամից մեծ վերջնական հաշիվների քանակը։

Ֆունկցիաներ՝ `analyze(const CafeBills&)`, `changeCopy(CafeBills object, int index, double value)`, `reversed(const CafeBills&)`։ Բնօրինակն արտածել յուրաքանչյուր կանչից հետո։ Փորձնական տվյալներ՝ amounts = {8000, 25000, 14000, 30000}, p = 10։

Տեսական մաս

Փոխանցում հղումով (`const CafeBills&`). պատճենում չկա, `const`-ը արգելում է փոփոխել օբյեկտը։ Այդպես են գրված `analyze` և `reversed`։ **Փոխանցում արժեքով**. `changeCopy`-ը ստանում է օբյեկտի *պատճենը* (կանչվում է լռելյայն պատճենող կոնստրուկտորը, զանգվածը պատճենվում է տարր առ տարր), ուստի փոփոխությունը չի ազդում բնօրինակի վրա։ **Օբյեկտի վերադարձ**. `reversed`-ը կառուցում է նոր `CafeBills` շրջված գումարներով և նույն `p`-ով. բնօրինակը մնում է նույնը։

Կոնստրուկտորը ստուգում է սահմանները և խախտման դեպքում նետում `invalid_argument`, այսինքն սխալ վիճակով օբյեկտ չի ստեղծվում։ Նույն ստուգումը կատարում է `setAmount`-ը։ Վերջնական հաշիվը հաշվվում է ըստ պահանջի ամեն անգամ, ուստի պահվող տվյալները միայն սկզբնական գումարներն են և `p`-ն։

Դասի և ֆունկցիաների նկարագիրը

Անդամ	Նշանակություն
`CafeBills(const double[], int, double)`	կոնստրուկտոր՝ ստուգում է քանակը (1–20), գումարները (> 0), p-ն (0–100)
`finalBill(i) const`	amount · (1 + p/100)
`total() / minIndex() / countAbove(limit) const`	երեք պահանջվող հաշվարկները
`setAmount(i, v)`	գումարի փոփոխում՝ ինդեքսի և արժեքի ստուգումով
`analyze(const CafeBills&)`	արտածում է երեք հաշվարկները

`changeCopy(CafeBills, int, double)`	փոփոխում և արտածում է միայն տեղային պատճենը
`reversed(const CafeBills&)`	վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով, նույն p-ով

Ծրագրի կոդը

cafe.cpp

```cpp
// Լաբորատոր աշխատանք 11 — CafeBills դաս (սրճարանի հաշիվներ)
#include <iostream>
#include <iomanip>
#include <stdexcept>
#ifdef _WIN32
#include <windows.h>
#endif
using namespace std;

const int MAX_TABLES = 20;
const double LIMIT = 20000;           // «մեծ հաշվի» շեմ

class CafeBills {
    double amounts[MAX_TABLES];       // սեղանների պատվերների գումարներ
    int n;
    double p;                         // սպասարկման տոկոս
    static void check(double v) {
        if (v <= 0) throw invalid_argument("Գումարը պետք է լինի > 0");
    }
public:
    CafeBills(const double a[], int count, double percent) : n(count), p(percent) {
        if (count < 1 || count > MAX_TABLES) throw invalid_argument("Սեղանների քանակը 1–20 է");
        if (percent < 0 || percent > 100) throw invalid_argument("p-ն պետք է լինի 0–100");
        for (int i = 0; i < n; i++) { check(a[i]); amounts[i] = a[i]; }
    }
    int size() const { return n; }
    double percent() const { return p; }
    double amount(int i) const { return amounts[i]; }
    double finalBill(int i) const { return amounts[i] * (1 + p / 100); }

    void setAmount(int i, double v) {
        if (i < 0 || i >= n) throw out_of_range("Սխալ ինդեքս");
        check(v);
        amounts[i] = v;
    }
    double total() const {                    // վերջնական հաշիվների գումար
        double s = 0;
        for (int i = 0; i < n; i++) s += finalBill(i);
        return s;
    }
    int minIndex() const {                    // ամենափոքր վերջնական հաշվով սեղան
        int best = 0;
        for (int i = 1; i < n; i++)
            if (finalBill(i) < finalBill(best)) best = i;
        return best;
    }
    int countAbove(double limit) const {      // limit-ից մեծ հաշիվների քանակ
        int k = 0;
        for (int i = 0; i < n; i++) if (finalBill(i) > limit) k++;
        return k;
    }
    void print(const char* title) const {     // 2 տող՝ գումարներ և վերջնական հաշիվներ
        cout << title << " (p = " << p << "%)\n  amount:";
        for (int i = 0; i < n; i++) cout << setw(10) << amount(i);
        cout << "\n  final: ";
        for (int i = 0; i < n; i++) cout << setw(10) << finalBill(i);
        cout << '\n';
    }
};
```

```cpp

void analyze(const CafeBills& b) {
    cout << "Վերջնական հաշիվների գումար: " << b.total()
         << "\nԱմենափոքր հաշվով սեղանի ինդեքս: " << b.minIndex()
         << "\n" << LIMIT << "-ից մեծ հաշիվների քանակ: " << b.countAbove(LIMIT) << '\n';
}

// Օբյեկտը փոխանցվում է ԱՐԺԵՔՈՎ՝ փոխվում է միայն տեղային պատճենը
void changeCopy(CafeBills object, int index, double value) {
    object.setAmount(index, value);
    object.print("Տեղային պատճեն");
}

// Նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով
CafeBills reversed(const CafeBills& b) {
    double a[MAX_TABLES];
    for (int i = 0; i < b.size(); i++) a[i] = b.amount(b.size() - 1 - i);
    return CafeBills(a, b.size(), b.percent());
}

int main() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
#endif
    cout << fixed << setprecision(2);
    try {
        const double data[] = {8000, 25000, 14000, 30000};
        CafeBills bills(data, 4, 10);

        bills.print("Բնօրինակ");
        cout << "\n-- analyze --\n"; analyze(bills);
        bills.print("\nԲնօրինակ analyze-ից հետո");

        cout << "\n-- changeCopy(bills, 1, 5000) --\n";
        changeCopy(bills, 1, 5000);
        bills.print("\nԲնօրինակ changeCopy-ից հետո");

        cout << "\n-- reversed --\n";
        CafeBills rev = reversed(bills);
        rev.print("Շրջված օբյեկտ");
        bills.print("\nԲնօրինակ reversed-ից հետո");
    } catch (const exception& e) {
        cout << "Սխալ՝ " << e.what() << '\n';
        return 1;
    }
    return 0;
}
```

Ծրագրի ելք

```
Terminal — g++ -std=c++17 -Wall -Wextra cafe.cpp -o cafe && ./cafe

Բնօրինակ (p = 10.00%)
  amount:   8000.00  25000.00  14000.00  30000.00
  final:    8800.00  27500.00  15400.00  33000.00

-- analyze --
Վերջնական հաշիվների գումար: 84700.00
Ամենափոքր հաշվով սեղանի ինդեքս: 0
20000.00-ից մեծ հաշիվների քանակ: 2

Բնօրինակ analyze-ից հետո (p = 10.00%)
  amount:   8000.00  25000.00  14000.00  30000.00
  final:    8800.00  27500.00  15400.00  33000.00

-- changeCopy(bills, 1, 5000) --
Տեղային պատճեն (p = 10.00%)
  amount:   8000.00   5000.00  14000.00  30000.00
  final:    8800.00   5500.00  15400.00  33000.00
```

```
Բնօրինակ changeCopy-ից հետո (p = 10.00%)
  amount:   8000.00  25000.00  14000.00  30000.00
  final:    8800.00  27500.00  15400.00  33000.00

-- reversed --
Շրջված օբյեկտ (p = 10.00%)
  amount:  30000.00  14000.00  25000.00   8000.00
  final:   33000.00  15400.00  27500.00   8800.00

Բնօրինակ reversed-ից հետո (p = 10.00%)
  amount:   8000.00  25000.00  14000.00  30000.00
  final:    8800.00  27500.00  15400.00  33000.00
```

Ձեռքով ստուգում (p = 10 → գործակից 1.1)

Ինդեքս	0	1	2	3	Արդյունք
Գումար	8000	25000	14000	30000	
Վերջնական հաշիվ	8800	27500	15400	33000	գումար՝ **84700**

Ամենափոքրը՝ 8800, ինդեքս **0**։ 20000-ից մեծ՝ 27500 և 33000, այսինքն **2** հաշիվ։ `changeCopy(bills, 1, 5000)`՝ պատճենում [1] = 5000 → 5500, բնօրինակում [1] մնում է 25000 → 27500։ `reversed`՝ {30000, 14000, 25000, 8000}, p = 10։ Ելքը համընկնում է։

Հանձնման ստուգաթերթ

Կետ	Արդյունք
Կոմպիլացիա առանց զգուշացումների	✔ g++ -Wall -Wextra
Թույլատրելի արժեքների ստուգում	✔ գումար > 0, 0 ≤ p ≤ 100, նետվում է invalid_argument
`changeCopy`-ը չի փոփոխում բնօրինակը	✔ փոխանցում արժեքով, բնօրինակն արտածված է
`reversed`-ը վերադարձնում է նոր օբյեկտ	✔ նույն p, բնօրինակը անփոփոխ
Բնօրինակն արտածված է յուրաքանչյուր կանչից հետո	✔ 4 անգամ

Եզրակացություն

Ստեղծվեց `CafeBills` դասը։ Փորձնական տվյալների համար վերջնական հաշիվների գումարը 84700 է, ամենափոքր հաշվով սեղանը՝ ինդեքս 0, 20000-ից մեծ հաշիվների քանակը՝ 2։ Ցուցադրվեց, որ արժեքով փոխանցված օբյեկտի փոփոխությունը չի ազդում բնօրինակի վրա, իսկ `reversed`-ը վերադարձնում է նոր օբյեկտ։